Clara Jessica-26040027

summarAIse: A Fully-Local AI Lecture-Notes Summariser

1. Project Purpose

summarAIse is a desktop app designed to summarize lecture notes into a short yet accurate summary and a study flashcard set. It's unique because everything is running on the user's own machine: the language model is hosted by Ollama, meaning there is no need for an Internet connection, a paid API, or an account! The tool does not store a user's notes on any cloud service, making it appropriate for private notes, exam preparation and unpublished work which should not be uploaded to a cloud service.

The project is useful because it takes a long time and repetitiveness to turn long notes into study material. summarAIse does this step automatically and allows the control of the output: the user selects the model, uploads one or more files, and gets a Markdown version of the summary which can be downloaded, and an Anki version for active-recall revision. It can import 14 different file formats, and is not limited to plain text, it can read images and text, which means you can use it on real lecture material that might be a mixture of PDFs, slide decks or documents.

Inputs and outputs:

Input: Single file(s) of any of the 14 supported formats (PDF, DOCX, PPTX, XLSX, XLS, TXT, MD, HTML, HTM, CSV, JSON, XML, EPUB, MSG).

Output: summary.md (a Markdown summary) and optionally a flashcards.apkg (Anki flashcard deck).

2. Tools and Libraries Used

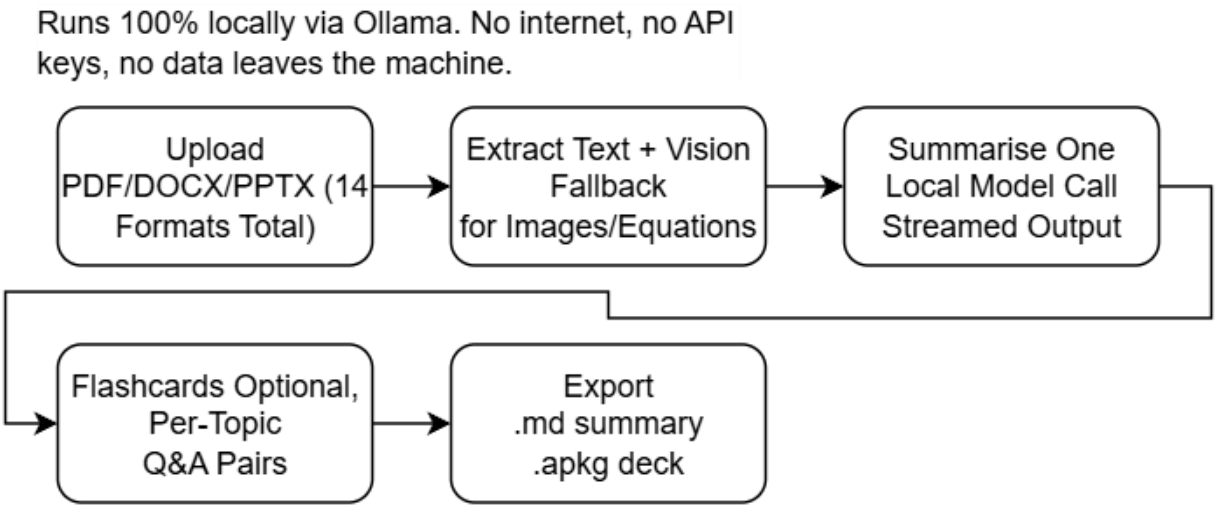
It is designed in Python and all the necessary libraries used are open source. Ollama is the local language model. The default model is qwen2.5vl:3b, a small vision capable model. The Python ollama client feeds the summary and passes images of the pages to the model for describing. The web interface is provided by Streamlit: Model picker, Multi-file uploader, Live streaming output, Download buttons.

Three libraries are used to extract texts, each optimised for its best formats: PyMuPDF is used for PDFs and supports per-page control and also provides a vision fallback, python-pptx for PowerPoint slides and MarkItDown for all other formats. Both pillow and the vision model are optimized to perform inference quickly, while pillow downscales the page images before they reach vision, and genanki creates the exportable Anki deck. The exact pinned versions can be found on requirements.txt.

3. Workflow

The application is designed around 5 stages of a pipeline. Firstly, one or more files are uploaded by the user. Secondly, every file is being extracted to text. When a page has no text or contains equations which cannot be copied well as text, the page is "rasterised" and its content is described with a vision model, which means that figures and equations are not lost. Thirdly, the whole text of all the files is passed to the chosen model in a single call and it generates the summary. The output is not displayed all at once but only continued as it is produced. Fourthly, the user can optionally turn the summary into flashcards. Fifth, the summary and the flash card deck can be exported. Each stage executes locally on user's machine.

Workflow diagram:



4. Code Explanation

This section will describe in a simple manner what each part of the code does (without line-by-line explanation). The project is divided into smaller files that are used to complete one task, making this easier to understand and maintain.

The main screen is app.py. This is the file you execute and creates the window that the user sees with Streamlit. It verifies when it starts that Ollama (the AI engine) is running and that there is at least one AI model installed, so the user won't be mystified by errors later. Then presents the dropdown for selection of model, and the area to drag files into. This file is the conductor. It gives the files to the reader, gathers the text, passes it to the file to be summarised, displays the resultant text on the screen, and provides the flashcard button. It is not the brainy guy that does the clever work but merely coordinates the other files.

The reader is ingest.py. It just opens any file the user uploaded, and extracts the words from that file. It has three different types of tools to deal with different types of files: one for PDFs, one for PowerPoint slides and a general one for all other files. The nifty part is when there's a page with no readable text, such as a scanned page or a page that is mostly a picture of an equation. Instead of giving up and losing that content, it takes a picture of the page and asks the vision model to describe what it sees, so nothing important is skipped.

The writer is summarise.py. This file takes all the text that was pulled out and turns it into the actual summary. It wraps the text in a series of instructions to the AI to summarise it, and then passes it to the local model using Ollama. The streamed summary is seen to appear on screen, since it is the user who is receiving it, but it is not produced all at once, instead, as the model thinks about it, so it streams the summary out word by word. It also attempts to restart the model a few times in case it stalls, which means that it doesn't automatically cause the app to crash if it happens once.

The study-helper is flashcards.py. This file can convert a summary into question and answer flash cards, once a summary is created. It prompts the AI to generate meaningful questions and answers based on the summary, and then stores them as an .apkg file, the file format Anki Study App recognizes. That file can be imported directly into Anki and the user can begin to revise.

It is useful to keep the settings in a single place: in **config.py**, you can change default settings like which AI model to use, which file types are permitted, etc., and in **prompts.py**, you can change the wording of the instructions you send to the AI without having to search through the code. This allows for easy tweaking of the AI without affecting the logic.

There are two helper scripts. **ollama_client.py** is a small middle-man script that communicates with Ollama, such as checking if it is up and running, or listing installed models. **setup_check.py** is a tiny script that can be run by the user before launching, to confirm that the AI engine is running and a model is installed. It's there to catch the most common set up mistakes early on and show a clear message, rather than a confusing crash.

5. Test Cases and Results

The application was tested with three representative cases covering its main paths. The figures below are taken from a sample run and should be updated to match your own test.

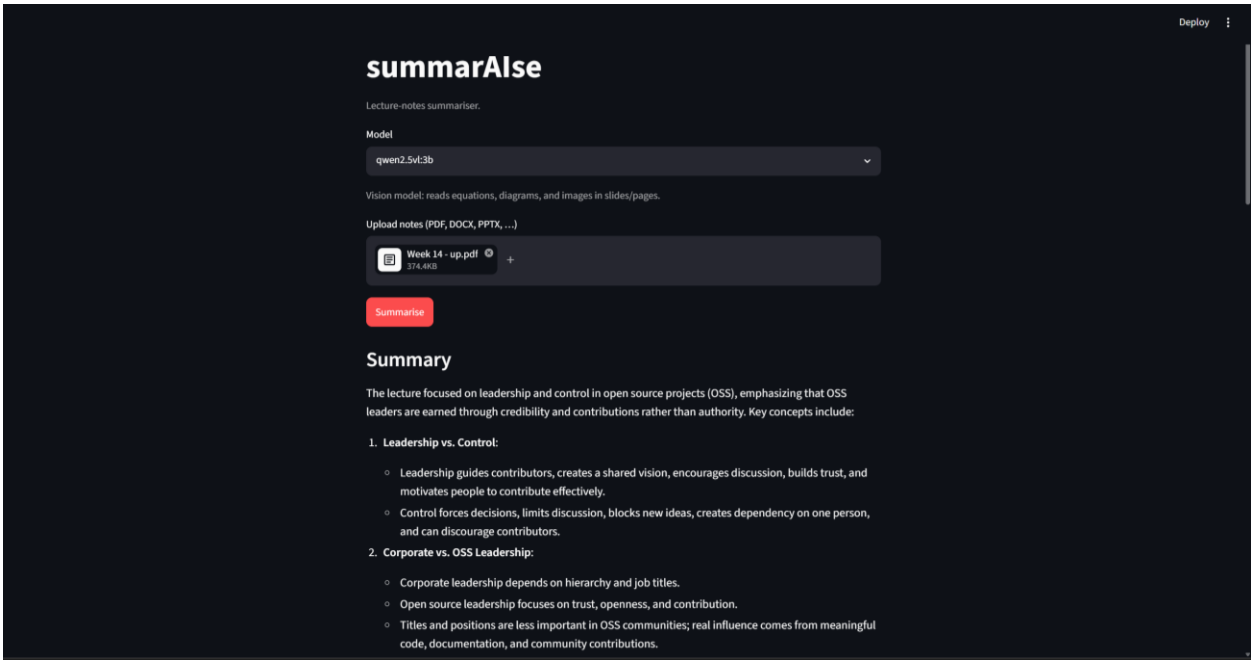
Case 1, text PDF: a 28-page lecture PDF was uploaded. The application produced a structured Markdown summary in about 4 minutes and 23 seconds on a CPU-only laptop, and the summary was downloaded successfully as summary.md. This exercises the fast text-only path.

Case 2, slide deck with images: a PowerPoint file containing equations stored as images was uploaded. The vision model described each figure, and the equations appeared in the resulting summary rather than being skipped, which confirms the vision-fallback path works.

Case 3, flashcards: using the summary from Case 1, the flashcard step generated 17 question-and-answer cards and exported a working flashcards.apkg file, which was then imported into the Anki application.

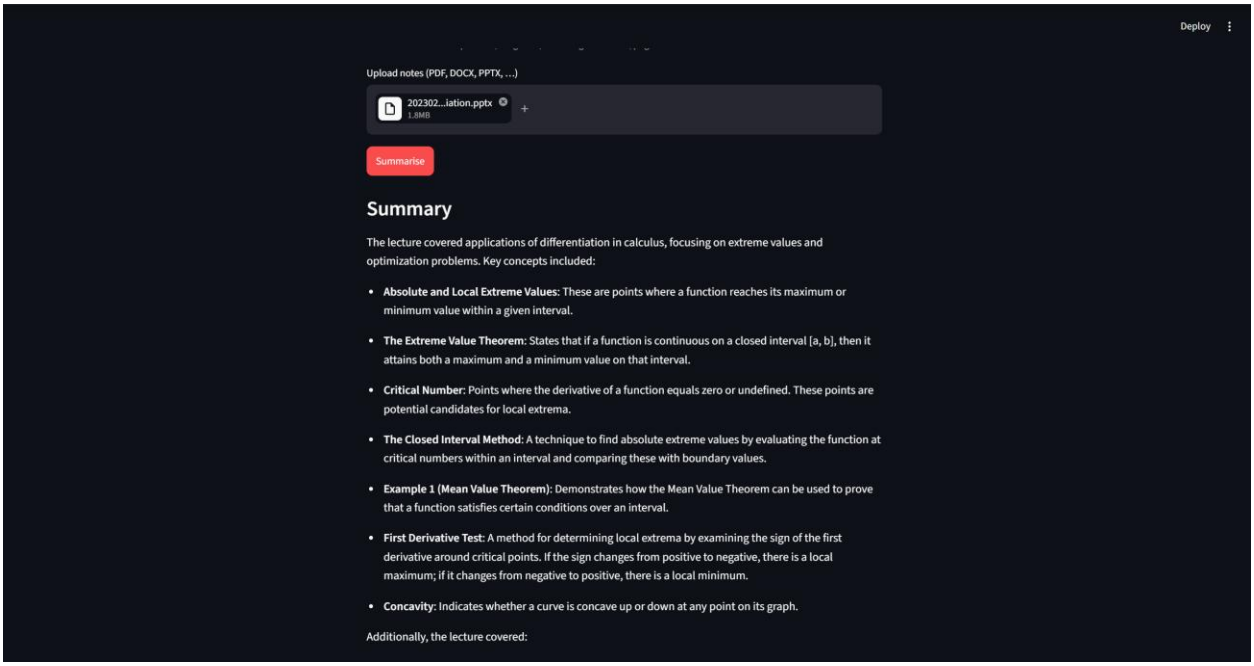
In addition, the helper script scripts/setup_check.py was run before launch. It confirmed that the Ollama server was running and that a model was installed, which prevents the most common setup errors.

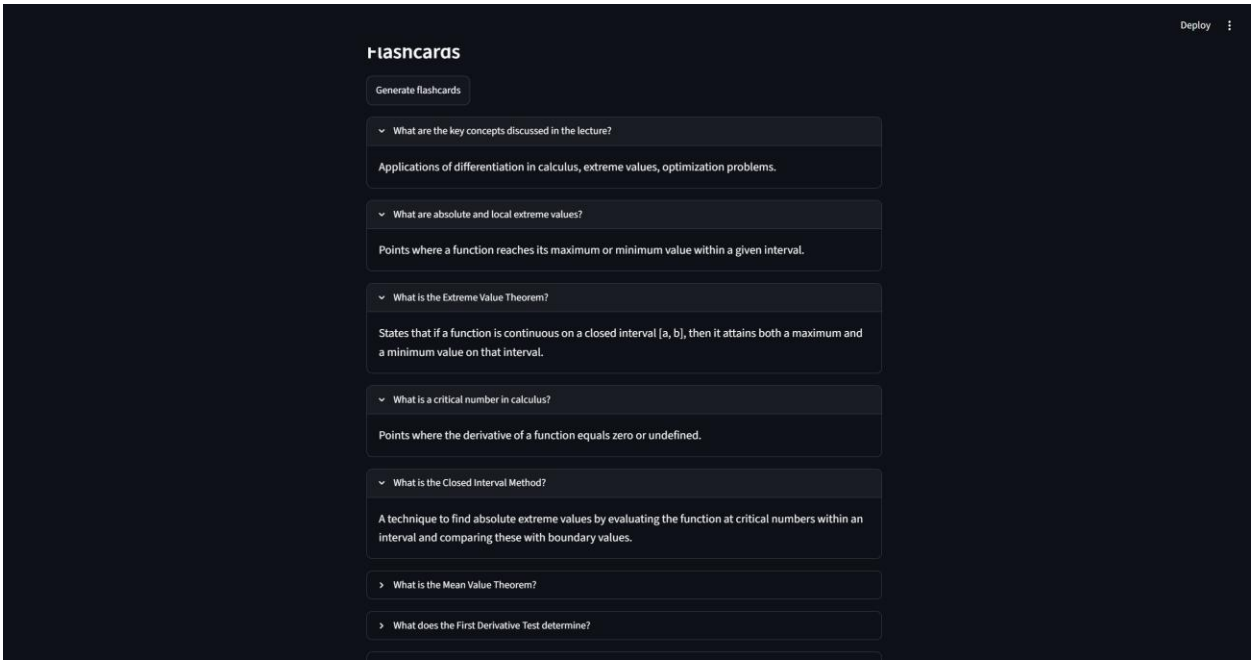
Sample output 1, generated summary in the app:



Clara Jessica-26040027

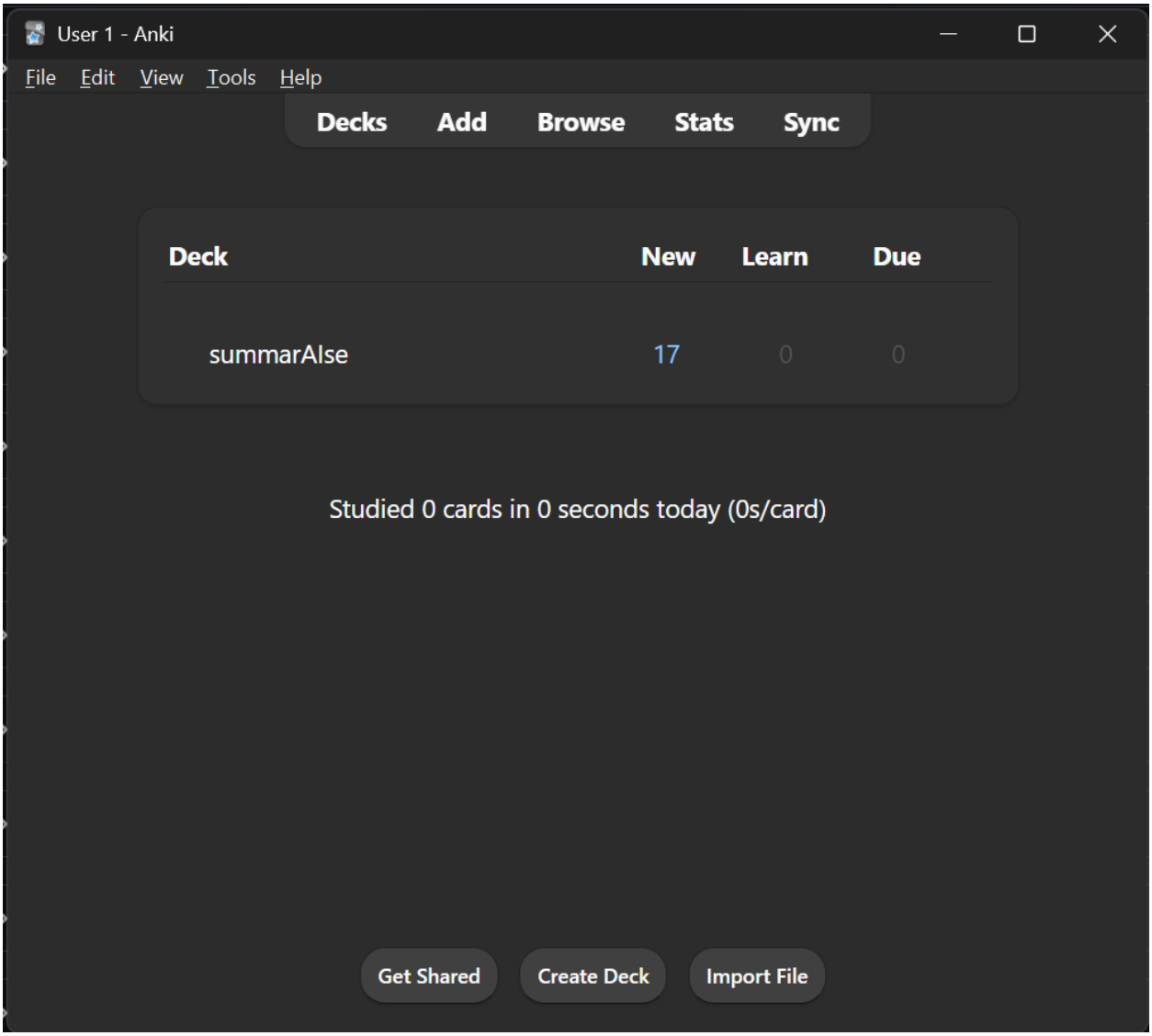
Sample output 2, generated flashcards:





Clara Jessica-26040027

Sample output 3, flashcards imported into Anki:



6. Importance of README.md and requirements.txt

The **README.md** is the starting point of a project that is open source. It explains to a new user what the project does, the requirements it has, how to install and run it, the licence and how to overcome common issues. If there is no README, then even if it is correct and well-written, it will be impossible for anyone else except its author to use it, as they don't know how to configure it or what it is used for. A good README will make it possible for other people to adopt and contribute to the project.

This package also contains a **requirements.txt** file that specifies the correct version and external libraries that the project is dependent on. Any user can repeat the same environment set up that the author used with one command: pip install -r requirements.txt. This makes the project reproducible: it works the same on another machine, rather than because library A is not installed or it exists in a different version.

In addition to a **LICENSE file** that specifies the legal conditions for reuse, these files allow a folder of code to become a project that others may install, run and build upon.